```
╔═══════════════════════════════════════════════════════════════════════╗
║              xKOR_3RR0R  —  FULL BUG REPORT & FIX GUIDE               ║
║              Compared against: github.com/gitsquared/edex-ui          ║
╚═══════════════════════════════════════════════════════════════════════╝

Last verified: May 2026
Repo analysed: https://github.com/krko2n/xKOR_3RR0R
Reference:     https://github.com/gitsquared/edex-ui

═══════════════════════════════════════════════════════════════════════
  ARCHITECTURE COMPARISON (your repo vs eDEX-UI)
═══════════════════════════════════════════════════════════════════════

eDEX-UI (reference):
  - All logic lives inside /src/
  - src/ has its OWN package.json with its own node_modules
  - node-pty is installed inside src/, not root
  - Font assets, sounds, themes are ALL bundled inside src/assets/
  - Entry point: src/_boot.js
  - Uses electron-rebuild only inside src/

xKOR_3RR0R (yours):
  - Root package.json only (no nested src/package.json)
  - Backend lives in /backend/, renderer in /src/renderer/
  - Assets folders exist but are EMPTY (only .gitkeep placeholder files)
  - Entry point: src/main.js → backend/server.js
  - Uses Express + WebSocket backend instead of IPC

Your architecture is fine and actually simpler. The problems are NOT structural —
they are 3 specific bugs listed below.

═══════════════════════════════════════════════════════════════════════
  BUG #1 — CRITICAL (app crashes on AI use)
  node-fetch is required but NOT installed
═══════════════════════════════════════════════════════════════════════

FILE:  backend/ai/proxy.js
LINE:  const fetch = require("node-fetch");   ← THIS CRASHES

PROBLEM:
  "node-fetch" is not listed in package.json dependencies.
  So after "npm install" it is never installed.
  When the AI panel sends a request, the backend crashes with:
    Error: Cannot find module 'node-fetch'

```

```
  IMPORTANT: Even if you add it, npm install gives you node-fetch v3 which is
  ESM-only — meaning require() still won't work. You'd get:
    Error [ERR_REQUIRE_ESM]: require() of ES Module not supported

  The correct fix is to DELETE the require entirely. Electron's main process
  runs on Node.js 18+ which has a BUILT-IN global fetch() — no package needed.

FIX — Edit backend/ai/proxy.js. Remove the require line completely:

  BEFORE (broken):
  ──────────────────────────────────────────────────
  const fetch = require("node-fetch");            ← DELETE THIS LINE
  const config = require("../../config/ai-endpoint.json");

  module.exports = async function (prompt) {
    try {
      const res = await fetch(config.endpoint, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ prompt }),
      });
      const data = await res.json();
      return data.response || "No response";
    } catch {
      return "AI endpoint unreachable";
    }
  };
  ──────────────────────────────────────────────────

  AFTER (fixed):
  ──────────────────────────────────────────────────
  const config = require("../../config/ai-endpoint.json");

  module.exports = async function (prompt) {
    try {
      const res = await fetch(config.endpoint, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ prompt }),
      });
      const data = await res.json();
      return data.response || "No response";
    } catch {
      return "AI endpoint unreachable";
    }
```

```
  };
  ──────────────────────────────────────────────────

  fetch() is now a global in Node.js 18+ — no import needed.

═══════════════════════════════════════════════════════════
  BUG #2 — CRITICAL (AI panel always fails silently)
  Content-Security-Policy blocks HTTP requests from renderer
═══════════════════════════════════════════════════════════

FILE:  src/renderer/index.html

PROBLEM:
  Your CSP (Content-Security-Policy) header says:
    connect-src ws://localhost:*

  This ONLY allows WebSocket connections (ws://).
  But ai.js does this:
    fetch("http://localhost:3001/ai", ...)   ← HTTP, not WS!

  The browser's security engine silently blocks this request.
  Result: AI panel always shows "AI endpoint unreachable"
  even when the backend and Ollama are running perfectly.

FIX — Edit the <meta> tag in src/renderer/index.html:

  BEFORE (broken):
  ──────────────────────────────────────────────────
  <meta http-equiv="Content-Security-Policy"
        content="default-src 'self';
                 connect-src ws://localhost:*;
                 script-src 'self';
                 style-src 'self' 'unsafe-inline';">
  ──────────────────────────────────────────────────

  AFTER (fixed):
  ──────────────────────────────────────────────────
  <meta http-equiv="Content-Security-Policy"
        content="default-src 'self';
                 connect-src ws://localhost:* http://localhost:*;
                 script-src 'self';
                 style-src 'self' 'unsafe-inline';
                 font-src 'self' https://fonts.gstatic.com;">
  ──────────────────────────────────────────────────

```

(The font-src addition is for Bug #3 below.)

BUG #3 — VISUAL (wrong/fallback font)
ShareTechMono font is referenced but never loaded

FILE: src/renderer/css/theme.css
LINE: font-family: 'ShareTechMono', monospace;

PROBLEM:
The font 'Share Tech Mono' is referenced everywhere in your CSS but:
- assets/fonts/ is empty (just a .gitkeep)
- There is no @font-face rule anywhere
- There is no Google Fonts <link> in index.html
- The old CSP blocked font CDNs anyway

Result: the browser silently falls back to your system monospace font.
The app works but looks wrong — not the intended cyberpunk style.

FIX — Add this line to src/renderer/index.html, inside <head>,
      BEFORE the CSS links:

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Share+Tech+Mono&display=swap" rel="stylesheet">
```

Also update the CSP (already shown in Bug #2 fix) to include:
  style-src 'self' 'unsafe-inline' https://fonts.googleapis.com;
  font-src 'self' https://fonts.gstatic.com;

STEP-BY-STEP: HOW TO FIX AND RUN xKOR_3RR0R

Prerequisites:
- Node.js 18 or newer (check: node --version)
- npm 9+   (check: npm --version)
- Git
- On Linux: build-essential, python3, libx11-dev, libxkbfile-dev, libsecret-1-dev
  Install with: sudo apt install build-essential python3 libx11-dev libxkbfile-dev libsecret-1-dev

STEP 1 — Clone the repo

```
  ──────────────────────────────────────────────────
  git clone https://github.com/krko2n/xKOR_3RR0R.git
  cd xKOR_3RR0R
  ──────────────────────────────────────────────────

STEP 2 — Fix Bug #1: remove require("node-fetch")
  Open:  backend/ai/proxy.js
  Find:  const fetch = require("node-fetch");
  Delete that entire line and save.

STEP 3 — Fix Bug #2: update the Content-Security-Policy
  Open:  src/renderer/index.html
  Find the <meta http-equiv="Content-Security-Policy" ...> tag.
  Replace the content attribute value with:

  "default-src 'self'; connect-src ws://localhost:* http://localhost:*; script-src 'self'; style-src 'sel
f' 'unsafe-inline' https://fonts.googleapis.com; font-src 'self' https://fonts.gstatic.com;"

STEP 4 — Fix Bug #3: load the font
  Open:  src/renderer/index.html
  Inside <head>, before any <link rel="stylesheet"> lines, add:

  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=Share+Tech+Mono&display=swap" rel="stylesheet">

STEP 5 — Install dependencies
  ──────────────────────────────────────────────────
  npm install
  ──────────────────────────────────────────────────
  This installs: electron, node-pty, ws, express, systeminformation, xterm.
  node-fetch is NOT needed anymore (we removed it in Step 2).

STEP 6 — Rebuild node-pty for your Electron version
  node-pty is a native module. It must be compiled specifically for
  the version of Electron you're using, NOT for plain Node.js.

  ──────────────────────────────────────────────────
  npm run postinstall
  ──────────────────────────────────────────────────

  This runs: electron-rebuild -f -w node-pty
  If it fails, try manually:
    npx @electron/rebuild -f -w node-pty

```

```
  Common error on Linux:
    "node_gyp_build: command not found"
  Solution:
    sudo apt install build-essential
    npm run postinstall

STEP 7 — (Optional) Configure AI endpoint
  If you want the AI panel to work, you need a local LLM running.
  The default config expects Ollama with llama3:

  Open:  config/ai-endpoint.json
  ──────────────────────────────────────────────
  {
    "endpoint": "http://localhost:11434/api/generate",
    "model": "llama3"
  }
  ──────────────────────────────────────────────

  To install Ollama: https://ollama.com/download
  Then run:  ollama pull llama3
  Then run:  ollama serve

  If you don't want AI, the app still works — the AI panel just won't respond.

STEP 8 — Launch the app
  ──────────────────────────────────────────────
  npm start
  ──────────────────────────────────────────────
  Or use the helper script:
  ──────────────────────────────────────────────
  bash run.sh
  ──────────────────────────────────────────────


═══════════════════════════════════════════════════════════════
  COMPLETE FINAL STATE OF FIXED FILES
═══════════════════════════════════════════════════════════════

── backend/ai/proxy.js (FIXED) ──────────────────────────────
const config = require("../../config/ai-endpoint.json");

module.exports = async function (prompt) {
  try {
    const res = await fetch(config.endpoint, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
```

```
      body: JSON.stringify({ prompt }),
    });
    const data = await res.json();
    return data.response || "No response";
  } catch {
    return "AI endpoint unreachable";
  }
};
```

```
── src/renderer/index.html <head> section (FIXED) ──────────────────────
<head>
    <meta charset="UTF-8">
    <title>xKOR_3RR0R</title>

    <meta http-equiv="Content-Security-Policy"
          content="default-src 'self';
                   connect-src ws://localhost:* http://localhost:*;
                   script-src 'self';
                   style-src 'self' 'unsafe-inline' https://fonts.googleapis.com;
                   font-src 'self' https://fonts.gstatic.com;">

    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Share+Tech+Mono&display=swap" rel="stylesheet">

    <link rel="stylesheet" href="css/boot.css">
    <link rel="stylesheet" href="css/theme.css">
    ... (rest unchanged)
</head>
```

```
═══════════════════════════════════════════════════════════════
  QUICK CHEAT SHEET (save this)
═══════════════════════════════════════════════════════════════

  Bug               File                        Fix
  ──────────────────────────────────────────────────────────────
  node-fetch crash  backend/ai/proxy.js         Delete require("node-fetch")
  CSP blocks AI     src/renderer/index.html     Add http://localhost:* to connect-src
  Font missing      src/renderer/index.html     Add Google Fonts <link> + CSP font-src
  ──────────────────────────────────────────────────────────────

  Commands to run after editing:
    npm install
```

```
    npm run postinstall    ← required every time Electron version changes
    npm start

═══════════════════════════════════════════════════════════
  WHAT STILL WORKS / WHAT'S NOT BROKEN
═══════════════════════════════════════════════════════════

  ✅ package.json — all dependencies except node-fetch are correct
  ✅ backend/server.js — Express + WebSocket setup is correct
  ✅ backend/terminal/pty.js — node-pty usage is correct
  ✅ src/main.js — Electron window setup is correct
  ✅ src/preload.js — contextBridge + WS reconnect logic is correct
  ✅ src/renderer/js/terminal.js — terminal session management is correct
  ✅ Globe, graphs, filemanager, keyboard — all fine structurally
  ✅ run.sh — correctly checks node_modules before starting

  ⚠️  Terminal output uses raw textContent (no xterm rendering)
     ANSI color codes will appear as raw text like "\u001b[32m"
     xterm is installed but never imported in the renderer.
     This is a future improvement, not a crash bug.

═══════════════════════════════════════════════════════════
  END OF GUIDE
═══════════════════════════════════════════════════════════
```